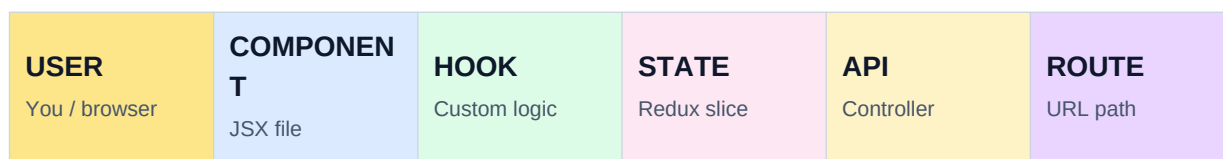


React 19 — E-commerce Frontend

Concepts, Code Walk-Through, Data Flow & Code Flow Guide

A walkthrough of every React concept used in this project — explained from scratch — followed by two kinds of diagram for each user journey: a **Data-Flow Diagram** (high-level: who triggers what) and a **Code-Flow Sequence Diagram** (low-level: which file calls which function in what order). Every diagram is paired with the actual code from the project plus a beginner-level walk-through.



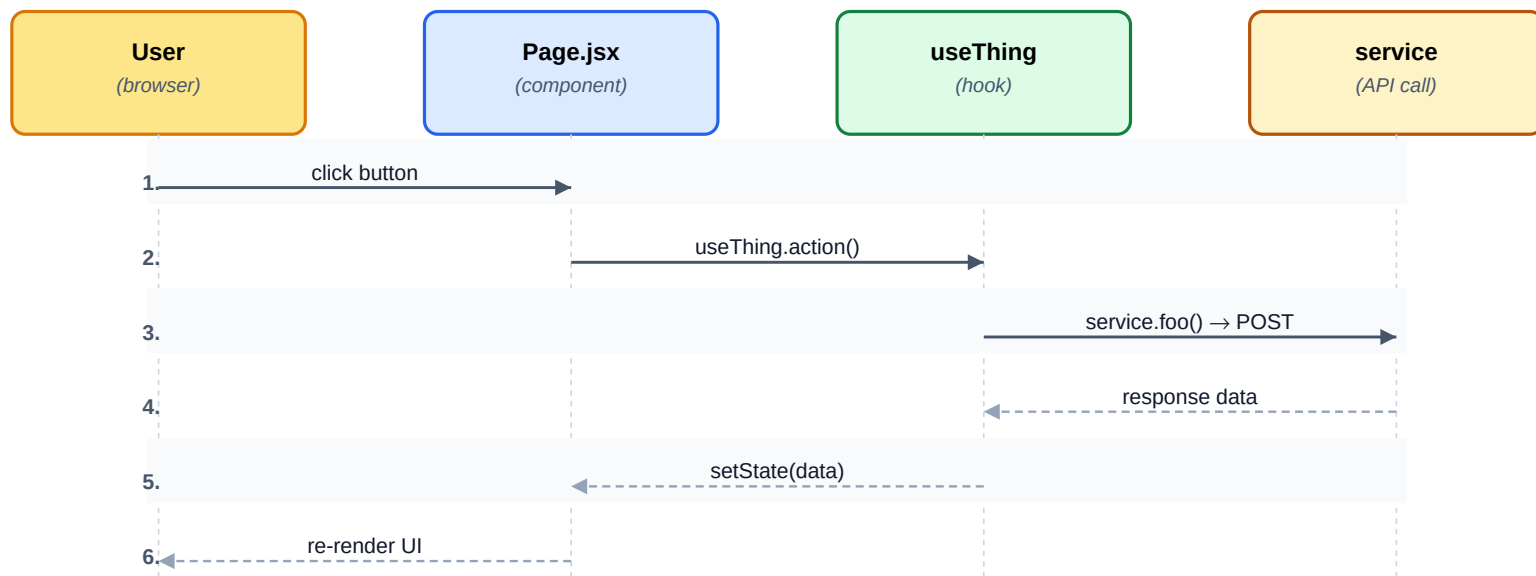
Each chapter contains:

1. Story-style narration of what happens.
2. **Data-flow diagram** (vertical line diagram).
3. **Code-flow sequence diagram** (file lanes + function-call arrows in order).
4. The actual code from the project (dark code box).
5. A yellow panel that walks through the code line by line.

API hops show only the controller name (e.g. *AuthController*) — never request/response bodies.

How to Read the Sequence Diagrams

A sequence diagram has TWO things: **actors** (the boxes at the top — one for each file or module involved) and **messages** (the horizontal arrows below — each one is a function call from the file on the left to the file on the right). Time flows top-down: step 1 happens first, step 2 next, and so on. A solid arrow means a normal call; a dashed arrow means a return value flowing back.



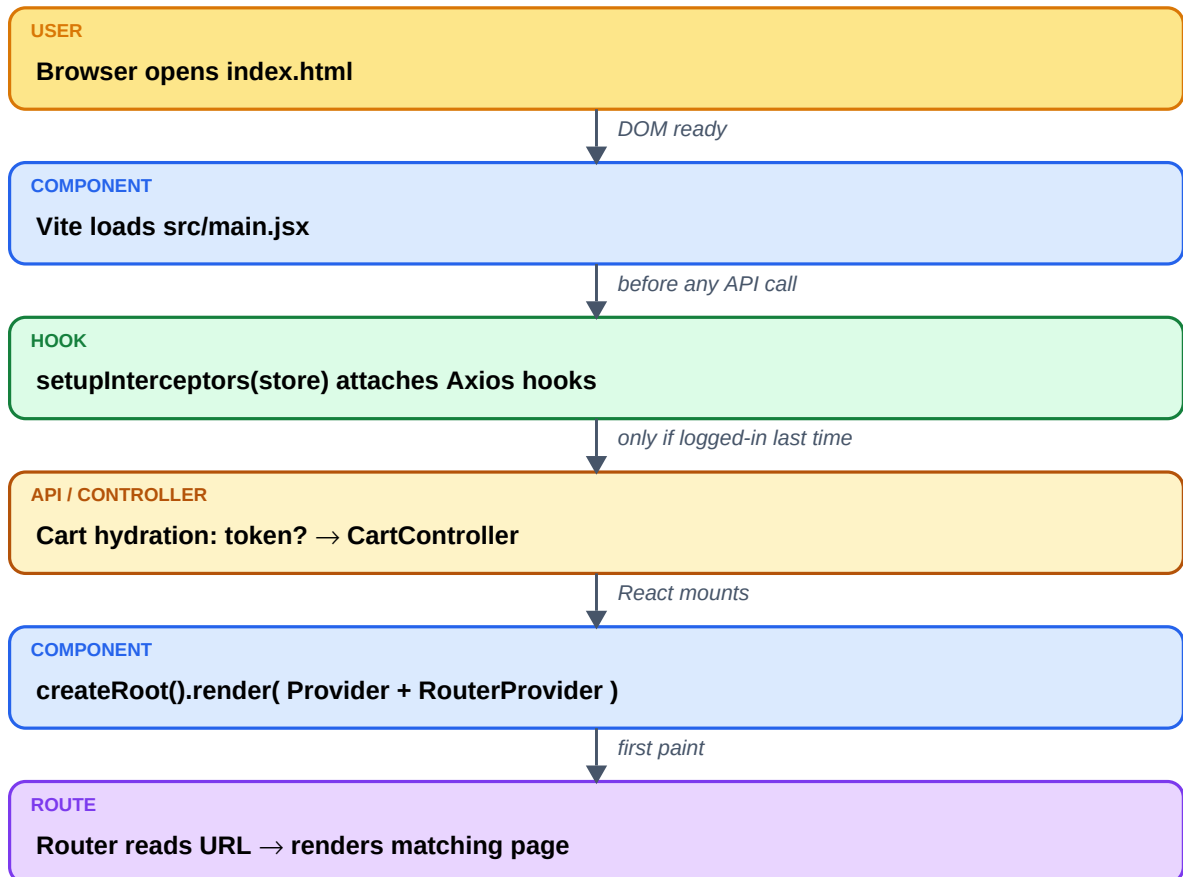
Sample sequence diagram. Solid arrows = calls; dashed arrows = returns. Step numbers on the left show order.

A sequence diagram is the BEST way to see **code flow** — i.e. exactly which file's function runs after which. If you ever get lost in the project, find the relevant sequence diagram in this guide, then open the named files side by side and follow the arrows.

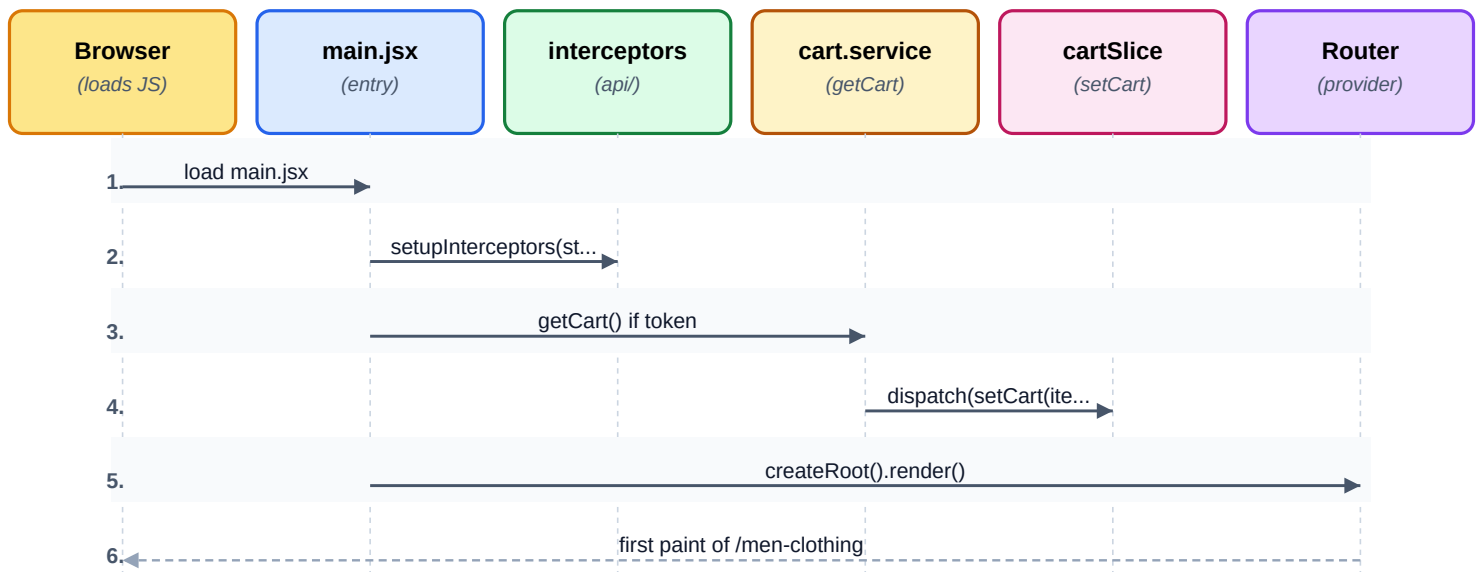
1. How a React App Starts

Browsers do not understand React. The browser loads `index.html` with an empty `<div id="root">`. Vite then runs `src/main.jsx`, and that file is where React wakes up and fills the div with our app.

Data flow — vertical view



Code flow — sequence view



Boot — five files cooperate before the first pixel is drawn.

Code

src/main.jsx

```

import { createRoot } from 'react-dom/client'
import { Provider } from 'react-redux'
import { RouterProvider } from 'react-router-dom'
import { store } from './store/store'
import { router } from './router/index'
import { setupInterceptors } from './api/interceptors'
import { getCart } from './features/cart/services/cart.service'
import { setCart } from './store/slices/cartSlice'

setupInterceptors(store)

if (localStorage.getItem('token')) {
  getCart()
    .then((items) => store.dispatch(setCart(items)))
}

createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <RouterProvider router={router} />
  </Provider>
)

```

What this code does — *how main.jsx wires the whole app together*

- **setupInterceptors(store)** runs first so every Axios call afterwards attaches the JWT and normalizes errors.
- If a JWT survived from a previous session, **getCart()** hits CartController and the result is dispatched into the cart slice.
- **createRoot(...).render(...)** mounts the React tree. Provider exposes Redux to every component; RouterProvider takes over the URL bar.

2. The Core React Vocabulary

Quick reference — every term used later.

Component

A function returning JSX. Filename matches function name in PascalCase.

Props

Read-only inputs: `<Child name='x' />`.

State (useState)

Per-component memory; setting it triggers a re-render.

Effect (useEffect)

Code that runs AFTER paint — used for fetching, navigation, logging.

Custom Hook

Reusable function starting with use; bundles related hooks together.

useMemo / useCallback

Cache derived values / function references to avoid wasted renders.

Context

Share a value with descendants without prop-drilling. Used by AuthContext.

Redux Slice

Global state outside the tree — auth and cart in this project.

useSelector / useDispatch

Read a slice value / send an action to mutate it.

Outlet, Navigate, useParams, useLocation, useNavigate

Router primitives — placeholder, redirect, URL data, programmatic nav.

lazy + Suspense

Code-split a component; show a spinner while loading.

ErrorBoundary

Class component that catches render errors and shows a fallback.

useActionState (React 19)

Form hook returning [state, formAction, isPending].

React Hook Form

Library to manage form fields + validation cleanly.

Axios Interceptors

Single-place rules for every HTTP call — auto JWT + error shape.

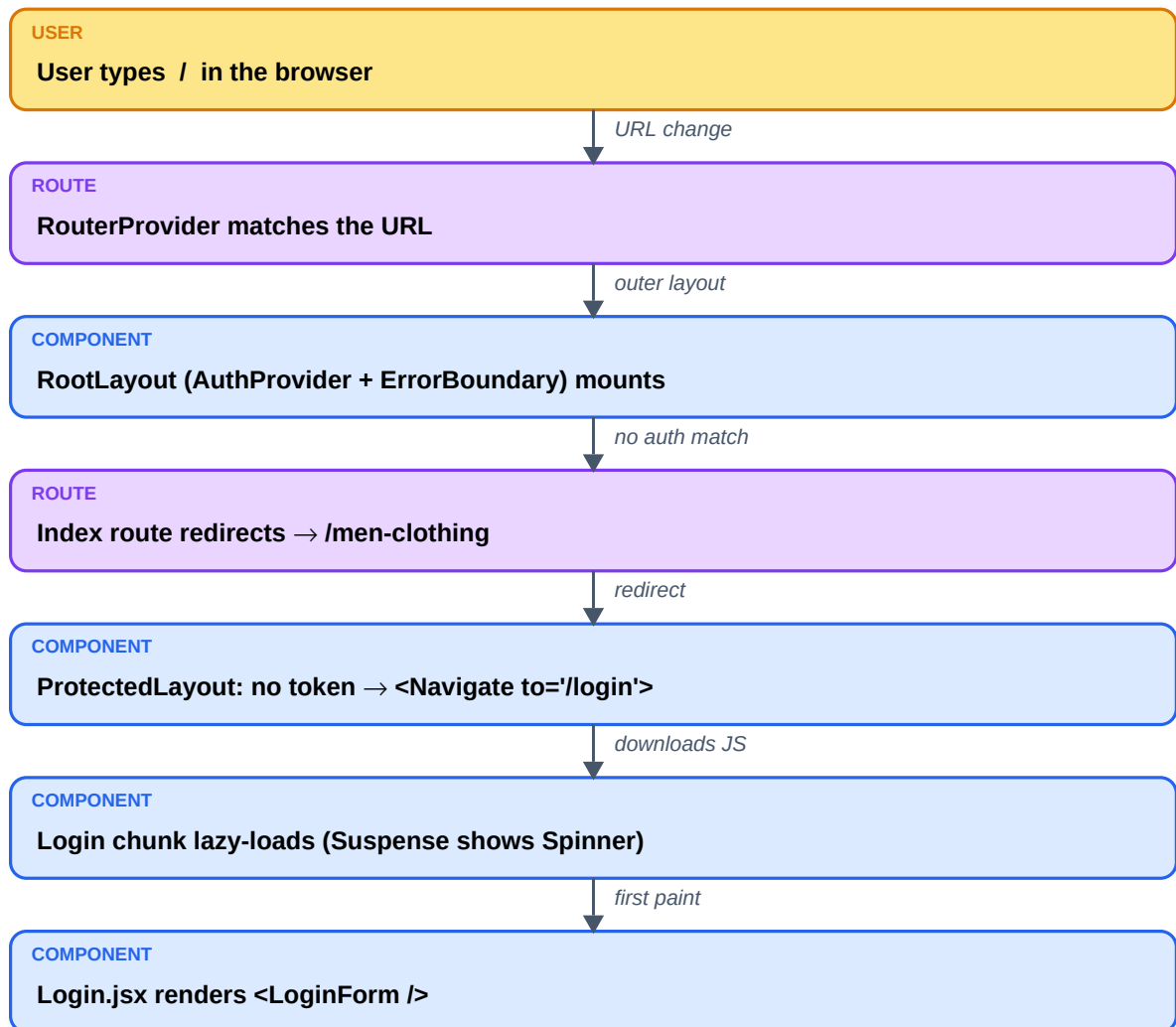
Optimistic UI

Update local state first, roll back if the server rejects.

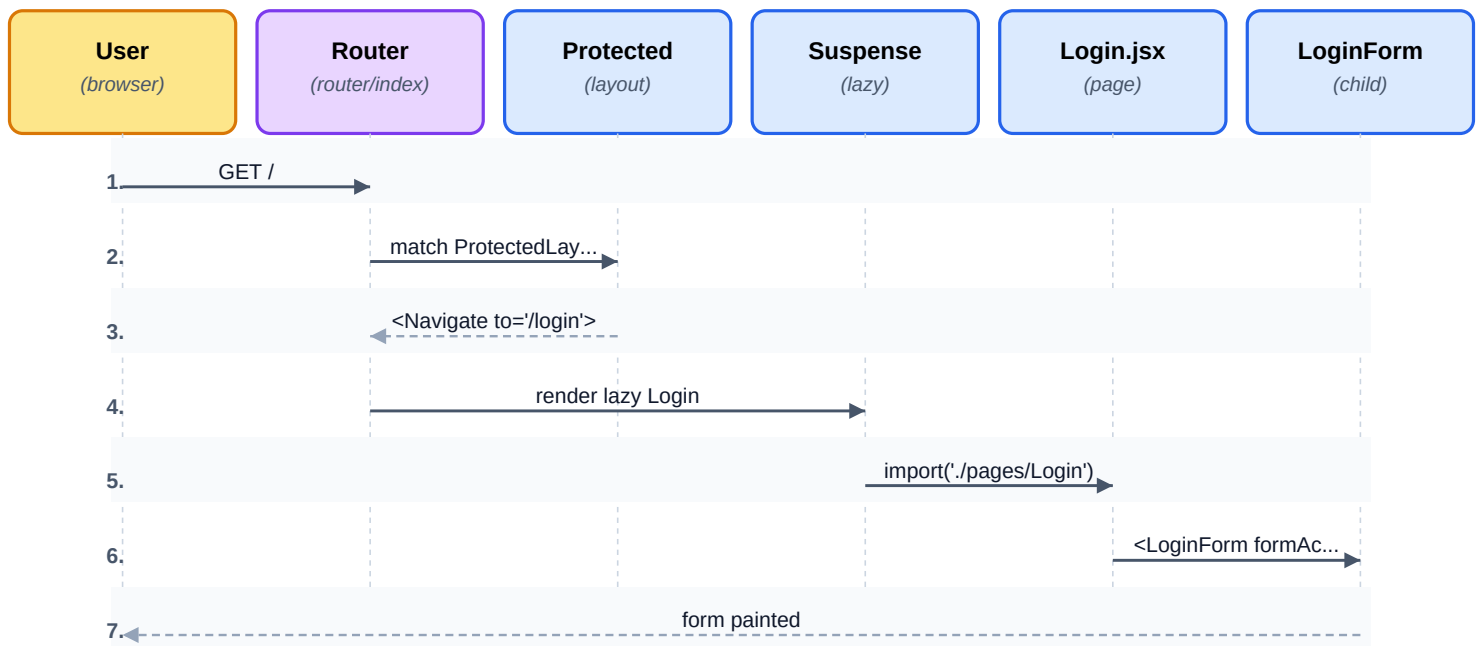
3. Customer Flow — Step 1: Loading the Login Page

User opens the site. There is no JWT, so the router sends them to /login. Login is lazy-loaded — its JS file is downloaded on demand.

Data flow



Code flow — sequence view



Loading Login — six files / hops between URL change and first form paint.

Code

src/features/auth/pages/Login.jsx

```
import { useLogin } from '../../hooks/useLogin'
import { LoginForm } from '../../components/LoginForm'

export default function Login() {
  const { state, formAction, isPending } = useLogin()
  return (
    <main>
      <div className={styles.card}>
        <h1>Welcome back</h1>
        <LoginForm formAction={formAction} state={state} isPending={isPending} />
      </div>
    </main>
  )
}
```

src/features/auth/components/LoginForm.jsx

```
export function LoginForm({ formAction, state, isPending }) {
  const errorMessage = state?.success === false ? state.error?.message : null
  return (
    <form action={formAction} noValidate>
      <input id="email" name="email" type="email" required disabled={isPending} />
      <input id="password" name="password" type="password" required disabled={isPending} />
      {errorMessage && <p role="alert">{errorMessage}</p>}
      <button type="submit" disabled={isPending}>
        {isPending ? 'Signing in...' : 'Sign In'}
      </button>
    </form>
  )
}
```

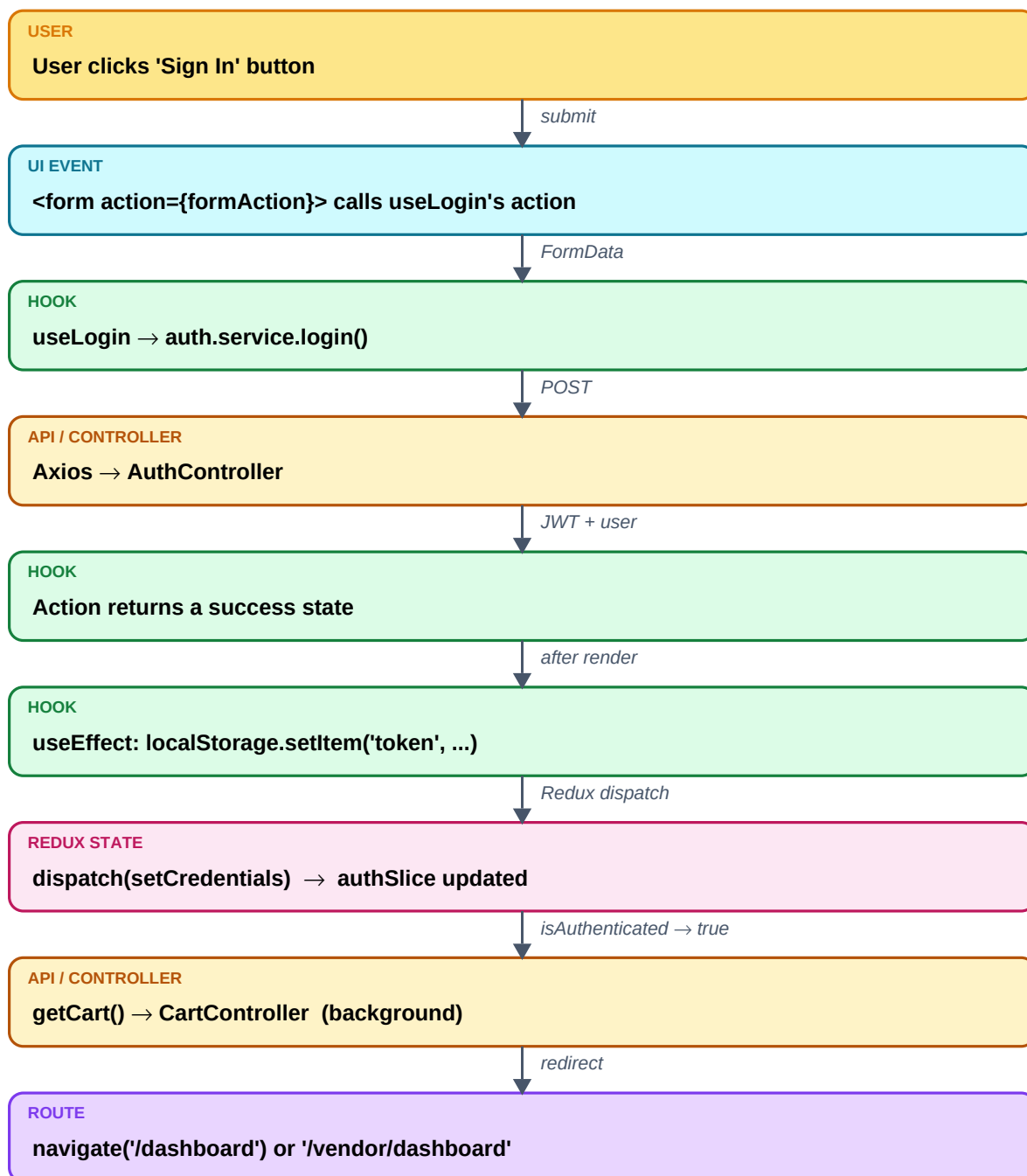
What this code does — *Login page + form — props and React 19 form action*

- **useLogin()** returns three things; we destructure them and pass each as a prop.
- **<form action={formAction}>** is React 19's native form action — no onSubmit handler needed.
- **disabled={isPending}** locks inputs while the API runs, preventing double submits.
- **{errorMessage && ...}** conditionally renders an alert when the previous submit failed.

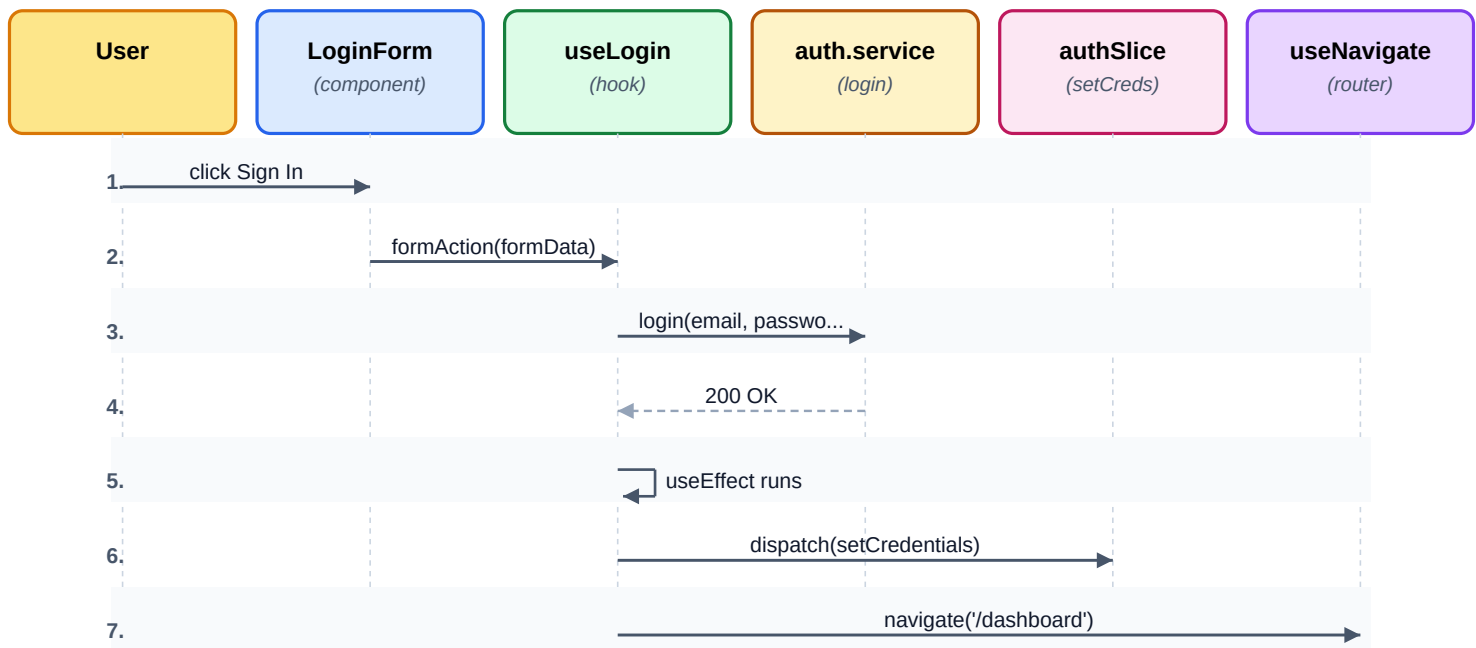
4. Customer Flow — Step 2: Submitting the Login Form

Clicking 'Sign In' must (1) call AuthController, (2) save JWT, (3) update Redux, (4) hydrate cart, (5) redirect. `useLogin` coordinates this with React 19's `useActionState`.

Data flow



Code flow — sequence view



Login submit — useLogin coordinates four files and one self-call (the post-state useEffect).

Code — useLogin

```
src/features/auth/hooks/useLogin.js

import { useActionState, useEffect } from 'react'
import { login } from '../../services/auth.service'
import { setCredentials } from '../../../store/slices/authSlice'
import { getCart } from '../../../cart/services/cart.service'
import { setCart } from '../../../store/slices/cartSlice'

export function useLogin() {
  const dispatch = useDispatch()
  const navigate = useNavigate()

  async function loginAction(prevState, formData) {
    const email = formData.get('email')?.trim()
    const password = formData.get('password')
    try {
      const result = await login({ email, password })
      return { success: true, data: result }
    } catch (err) {
      return { success: false, error: err }
    }
  }

  const [state, formAction, isPending] = useActionState(loginAction, null)

  useEffect(() => {
    if (!state?.success) return
    const { token, user } = state.data
    localStorage.setItem('token', token)
    dispatch(setCredentials({ token, user }))

    getCart().then((items) => dispatch(setCart(items)))

    const isVendor = user.role?.toLowerCase() === ROLES.VENDOR
    navigate(isVendor ? VENDOR_ROUTES.DASHBOARD : ROUTES.DASHBOARD)
  }, [state, dispatch, navigate])

  return { state, formAction, isPending }
}
```

What this code does — *useLogin* — three hooks working together

- **useActionState(loginAction, null)** wires the async action to `<form action={formAction}>`; React drives `isPending`.
- Inside the action we **ONLY** return — we do not navigate or write to `localStorage` there, because the new state has not been committed yet.
- **useEffect** runs **AFTER** React paints the new state. That is where side-effects belong: `localStorage` write, Redux dispatch, `navigate`.
- Cart hydration happens in the background; if it fails the 401 interceptor logs the user out anyway.

Code — auth service + Redux slice

src/features/auth/services/auth.service.js

```
import api from '../../../../api/axios.instance'

// POST /api/auth/login → returned to the caller
export async function login({ email, password }) {
  const response = await api.post('/auth/login', { email, password })
  return response.data
}
```

src/store/slices/authSlice.js

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  token: localStorage.getItem('token') ?? null,
  user: JSON.parse(localStorage.getItem('user') ?? 'null'),
}

const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    setCredentials(state, action) {
      state.token = action.payload.token
      state.user = action.payload.user
    },
    logout(state) {
      state.token = null; state.user = null
    },
  },
})

export const { setCredentials, logout } = authSlice.actions
export const selectCurrentUser = (state) => state.auth.user
export const selectIsAuthenticated = (state) => !!state.auth.token
export default authSlice.reducer
```

What this code does — *Service + slice — clear separation of concerns*

- **auth.service.js** is the only file that knows the URL /auth/login. The rest of the app stays clean.
- It uses the shared Axios instance — interceptors automatically add the JWT and normalize errors.
- **createSlice** generates a reducer + action creators in one block. We can write `state.token = ...` as if mutating, because Redux Toolkit uses Immer under the hood.
- **initialState** reads from `localStorage` so a hard refresh keeps the user logged in.
- **Selectors** are functions `(state) => ....` Components call them via `useSelector(selectIsAuthenticated)`.

Code — the Axios interceptors

```
src/api/interceptors.js

export function setupInterceptors(store) {
  // Request: attach JWT before sending
  api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token')
    if (token) config.headers.Authorization = `Bearer ${token}`
    return config
  })

  // Response: normalize errors + auto-logout on 401
  api.interceptors.response.use(
    (response) => response,
    (error) => {
      const normalized = normalizeError(error)
      if (normalized.status === 401 && store) {
        localStorage.removeItem('token')
        localStorage.removeItem('user')
        store.dispatch(logout())
      }
      return Promise.reject(normalized)
    },
  )
}
```

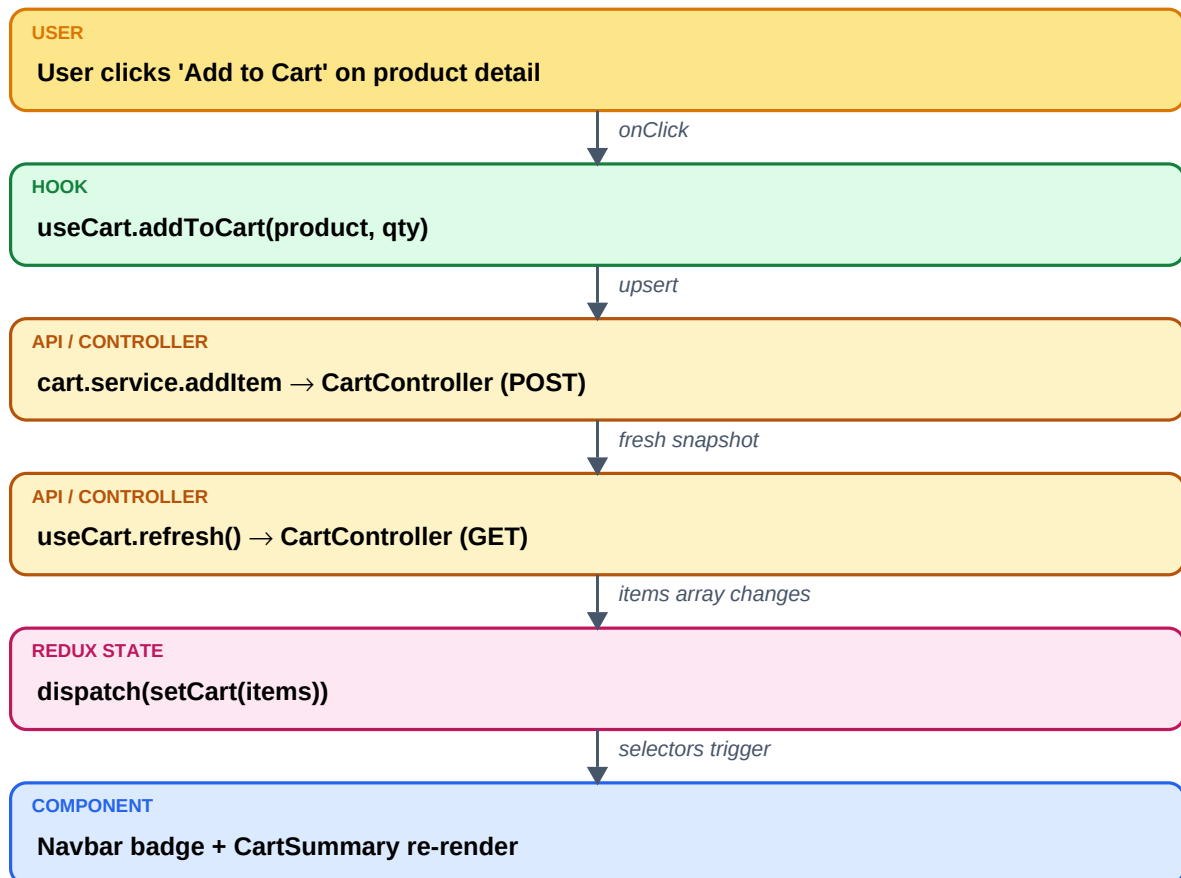
What this code does — *Interceptors* — write once, applied to every API call

- The **request interceptor** runs BEFORE the HTTP call leaves the browser. It reads the token and stamps the Authorization header.
- The **response interceptor** normalizes errors into a clean shape so all components handle them the same way.
- If the server returns 401, we wipe localStorage and dispatch logout(). Every component using `selectIsAuthenticated` instantly re-renders and lands on /login.
- Set up ONCE in main.jsx — no service file ever has to add headers or handle 401 itself.

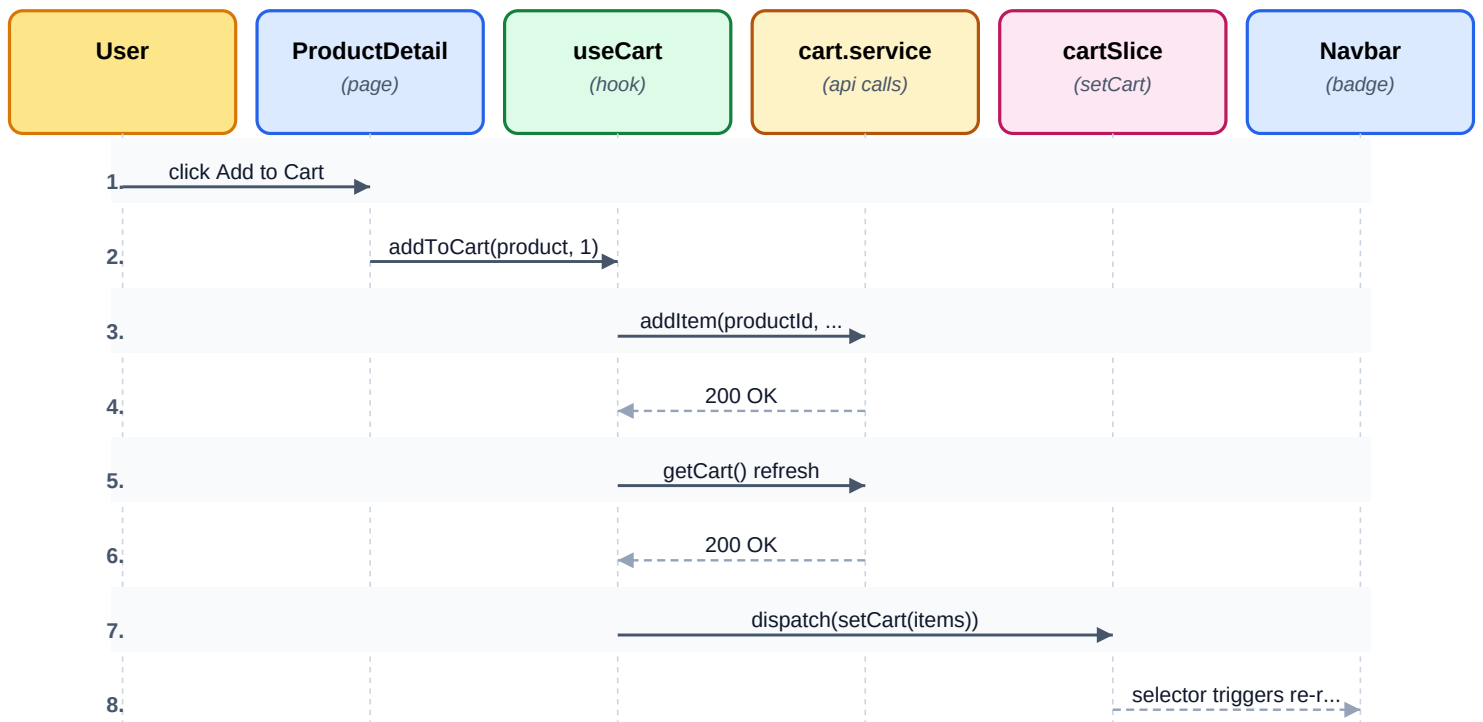
5. Customer Flow — Step 3: Browsing & Adding to Cart

On the product detail page, clicking 'Add to Cart' calls `useCart.addToCart`. Every cart mutation is followed by a re-fetch so Redux stays in sync with the server.

Data flow



Code flow — sequence view



Add to Cart — mutate, then re-fetch, then dispatch. The Navbar badge updates automatically.

Code

src/features/cart/hooks/useCart.js (key parts)

```
import { useDispatch, useSelector } from 'react-redux'
import { setCart, selectCartItems, selectCartCount,
  selectCartSubtotal } from '../../../../store/slices/cartSlice'
import * as cartApi from '../../../../services/cart.service'

export function useCart() {
  const dispatch = useDispatch()
  const items = useSelector(selectCartItems)
  const count = useSelector(selectCartCount)
  const subtotal = useSelector(selectCartSubtotal)

  async function refresh() {
    const fresh = await cartApi.getCart()
    dispatch(setCart(fresh))
  }

  async function addToCart(product, quantity = 1) {
    await cartApi.addItem({ productId: product.id, quantity })
    await refresh()
  }

  async function removeFromCart(cartItemId) {
    await cartApi.removeItem(cartItemId); await refresh()
  }

  async function updateQuantity(cartItemId, quantity) {
    if (quantity <= 0) await cartApi.removeItem(cartItemId)
    else await cartApi.updateItem(cartItemId, quantity)
    await refresh()
  }

  return { items, count, subtotal, addToCart, removeFromCart, updateQuantity }
}
```

What this code does — *useCart* — every mutation does the same dance

- Every action sends an API request, then re-fetches the whole cart, then dispatches `setCart(fresh)` — server is the single source of truth.
- `useSelector` subscribes the component to items; selectors compute count and subtotal automatically.
- Components never see Axios or Redux directly — they just call `addToCart(product)`.

Code — the cart slice

src/store/slices/cartSlice.js

```
import { createSlice } from '@reduxjs/toolkit'

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    setCart(state, action) { state.items = action.payload },
    clearCart(state) { state.items = [] },
  },
})

export const { setCart, clearCart } = cartSlice.actions

export const selectCartItems = (s) => s.cart.items
export const selectCartCount = (s) => s.cart.items.reduce((n, i) => n + i.quantity, 0)
export const selectCartSubtotal = (s) => s.cart.items.reduce((n, i) => n + i.price * i.quantity, 0)
export const selectCartIsEmpty = (s) => s.cart.items.length === 0

export default cartSlice.reducer
```

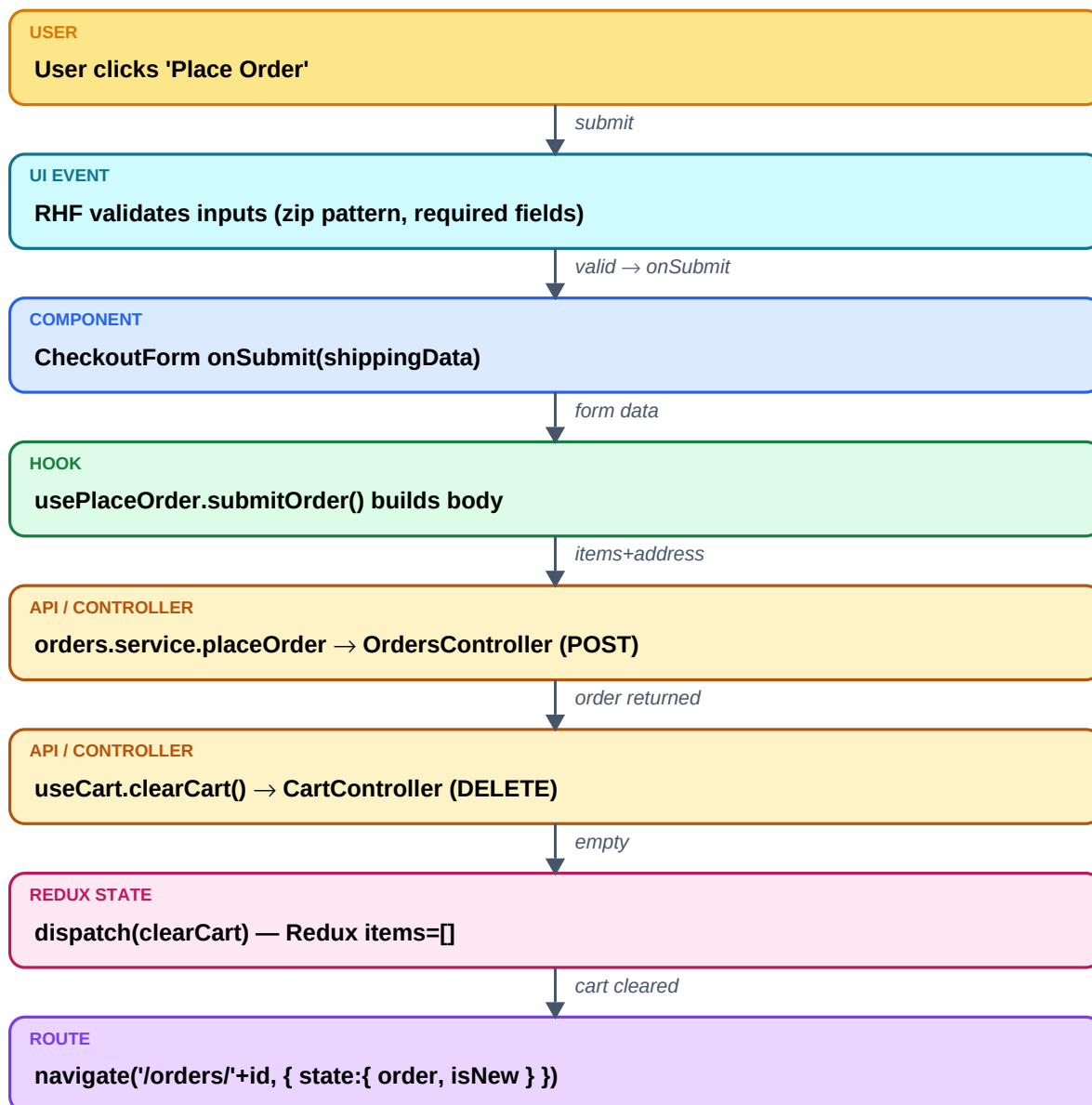
What this code does — *cartSlice* — *single source of truth for the badge*

- **setCart** replaces the entire array with a server snapshot — used after every mutation.
- **clearCart** is a local-only blank — used after place-order to empty the badge instantly.
- **Selectors** take the whole Redux state and return a piece of it (or compute one). Referentially stable, so React re-renders only when the value actually changes.
- When `useSelector(selectCartCount)` runs, Redux subscribes that component to `state.cart.items`. Any change recomputes the count and re-paints the component.

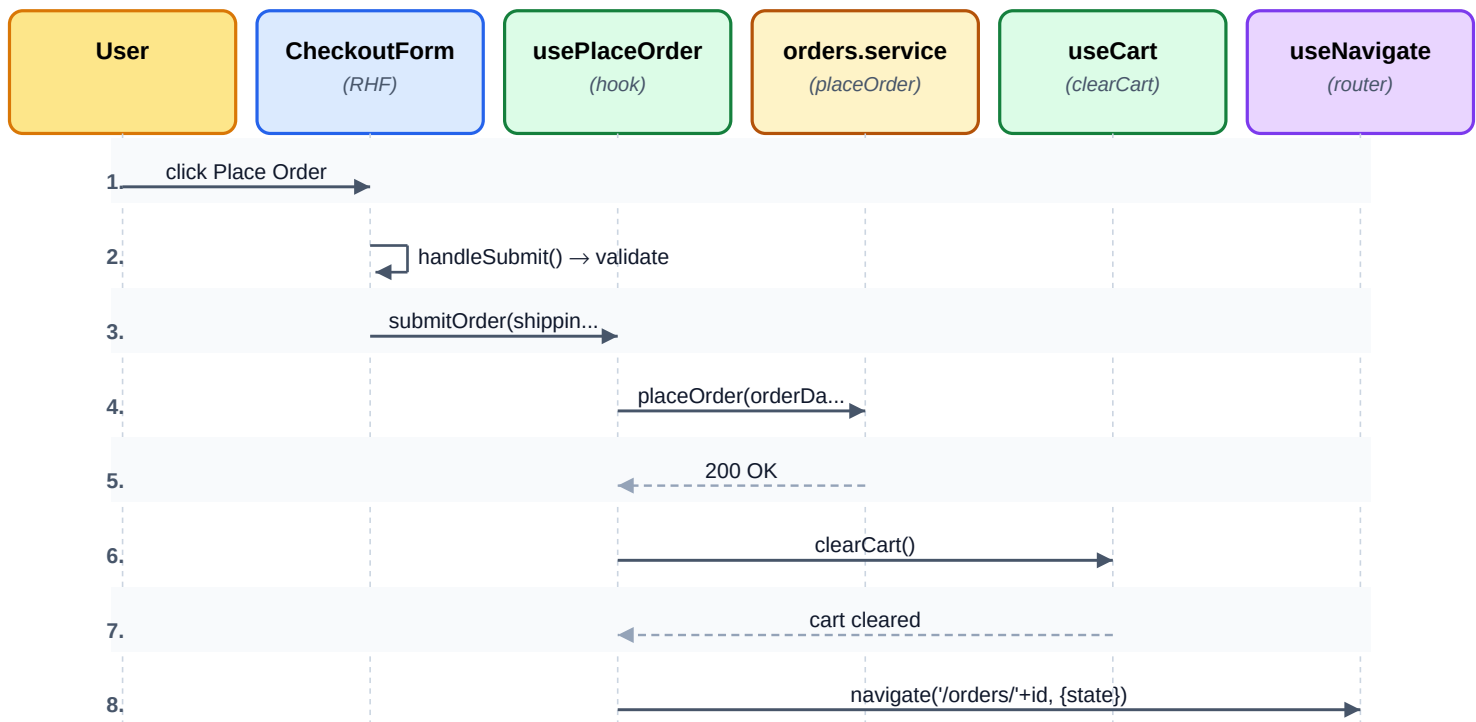
6. Customer Flow — Step 4: Checkout & Placing an Order

On /checkout the user fills shipping fields. **React Hook Form** validates them. On valid submit, `usePlaceOrder.submitOrder` POSTs to `OrdersController`, clears the cart, and navigates to the new order's page.

Data flow



Code flow — sequence view



Place Order — RHF validates, then a POST + DELETE + navigate.

Code — CheckoutForm with React Hook Form

src/features/checkout/components/CheckoutForm.jsx (excerpt)

```
import { useForm } from 'react-hook-form'

export function CheckoutForm({ onSubmit, isLoading }) {
  const { register, handleSubmit, formState: { errors } } = useForm({ mode: 'onTouched' })

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      <input {...register('firstName', { required: 'Required' })} />
      {errors.firstName && <span>{errors.firstName.message}</span>}

      <input {...register('zip', {
        required: 'Required',
        pattern: { value: /^[^d{5}(-\d{4})?$/], message: 'Invalid ZIP' },
      })} />
      {errors.zip && <span>{errors.zip.message}</span>}

      <button type="submit" disabled={isLoading}>
        {isLoading ? 'Placing order...' : 'Place Order'}
      </button>
    </form>
  )
}
```

What this code does — *CheckoutForm* — *React Hook Form in three lines*

- **useForm()** returns three tools: register (wires an input), handleSubmit (wraps your submit handler with validation), formState.errors (validation messages).
- **{...register('zip', { ... })}** spreads the props the input needs. RHF tracks the value internally — no useState for each field.
- **handleSubmit(onSubmit)** only calls onSubmit if every field passes validation; otherwise it populates errors and re-renders.
- **mode: 'onTouched'** — validation runs after the user blurs a field for the first time, friendlier than validating on every keystroke.

Code — usePlaceOrder hook

```
src/features/orders/hooks/usePlaceOrder.js

export function usePlaceOrder() {
  const [isLoading, setIsLoading] = useState(false)
  const [error, setError] = useState(null)
  const { items, subtotal, clearCart } = useCart()
  const navigate = useNavigate()

  async function submitOrder(shippingData) {
    setIsLoading(true); setError(null)
    const tax = subtotal * TAX_RATE
    const orderData = {
      items: items.map((i) => ({
        productId: i.productId, name: i.name,
        price: i.price, quantity: i.quantity, image: i.image,
      })),
      subtotal, shipping: 0,
      tax: parseFloat(tax.toFixed(2)),
      total: parseFloat((subtotal + tax).toFixed(2)),
      shippingAddress: { ...shippingData, country: 'India' },
    }
    try {
      const order = await placeOrder(orderData)
      await clearCart()
      navigate(ROUTES.ORDER_DETAIL(order.id),
        { state: { order, isNew: true } })
    } catch (err) {
      setError(err)
    } finally {
      setIsLoading(false)
    }
  }
  return { submitOrder, isLoading, error }
}
```

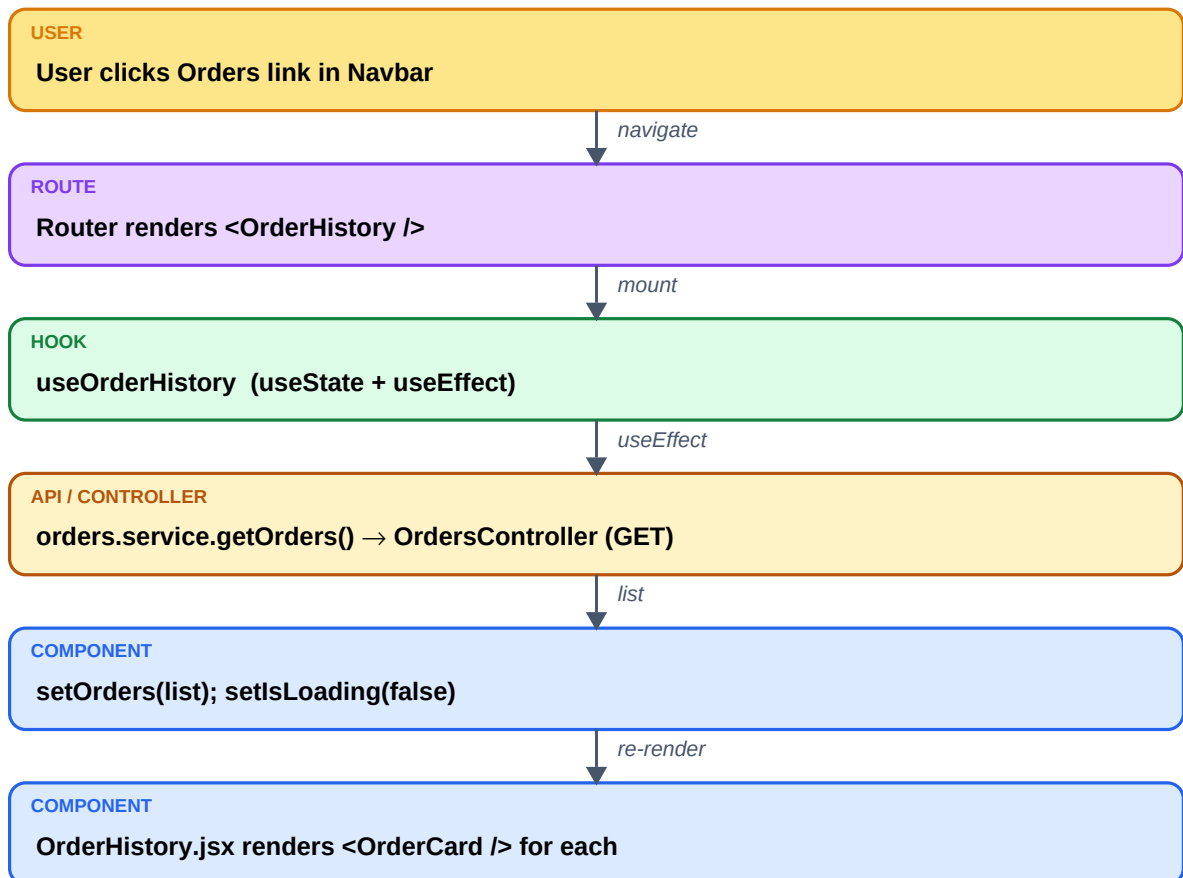
What this code does — *usePlaceOrder* — *composing two hooks*

- Calls `useCart()` to read items + subtotal AND to clear the cart after success — hooks composing other hooks is the React way.
- `{ ...shippingData, country: 'India' }` spreads the form fields and adds a default country.
- `navigate(path, { state: { order, isNew: true } })` passes the freshly persisted order to the destination page so it does not need a second GET.
- `try / catch / finally` keeps `isLoading` in sync regardless of outcome.

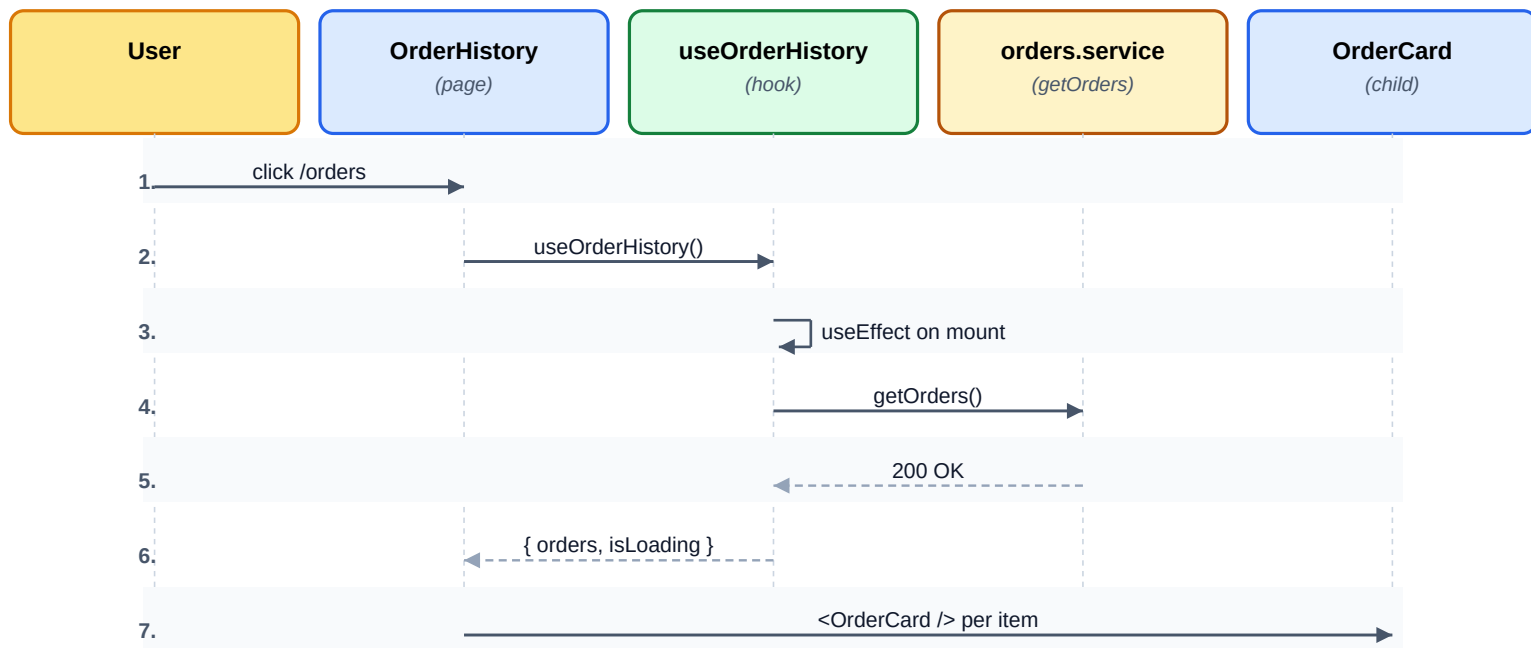
7. Customer Flow — Step 5: Viewing the Orders Page

`/orders` shows the history; `/orders/:id` shows one order. Both follow the canonical **useState + useEffect** server-fetch pattern.

Data flow — orders list

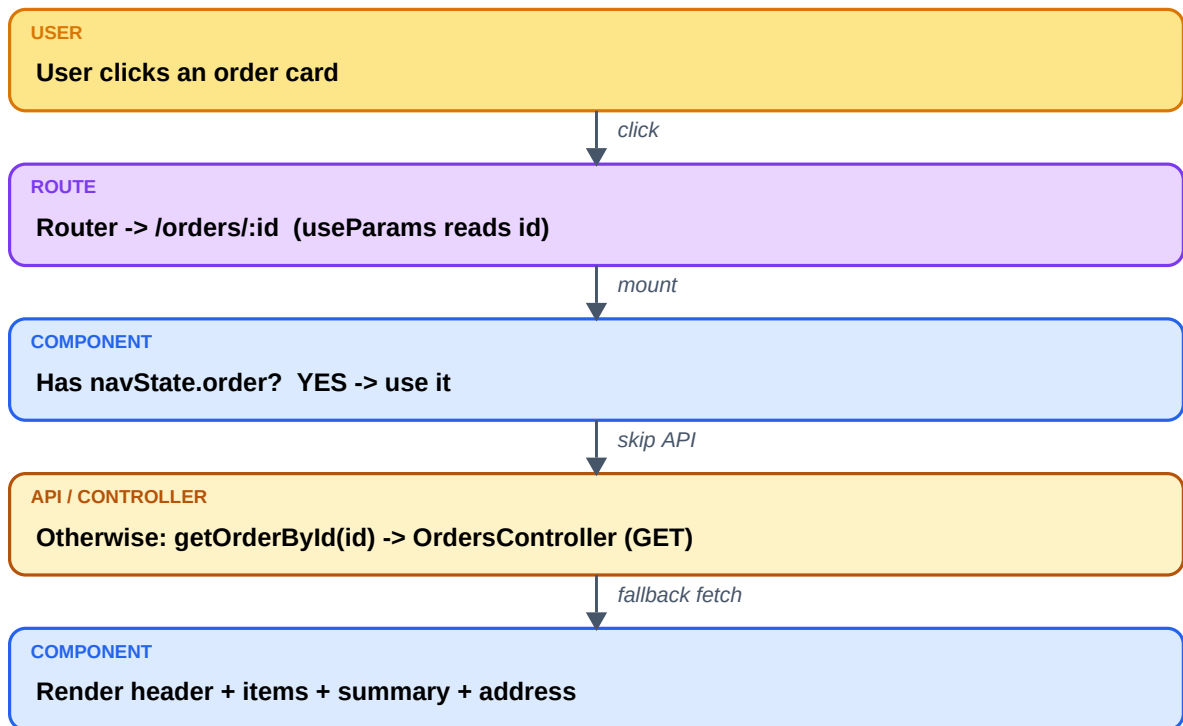


Code flow — sequence view

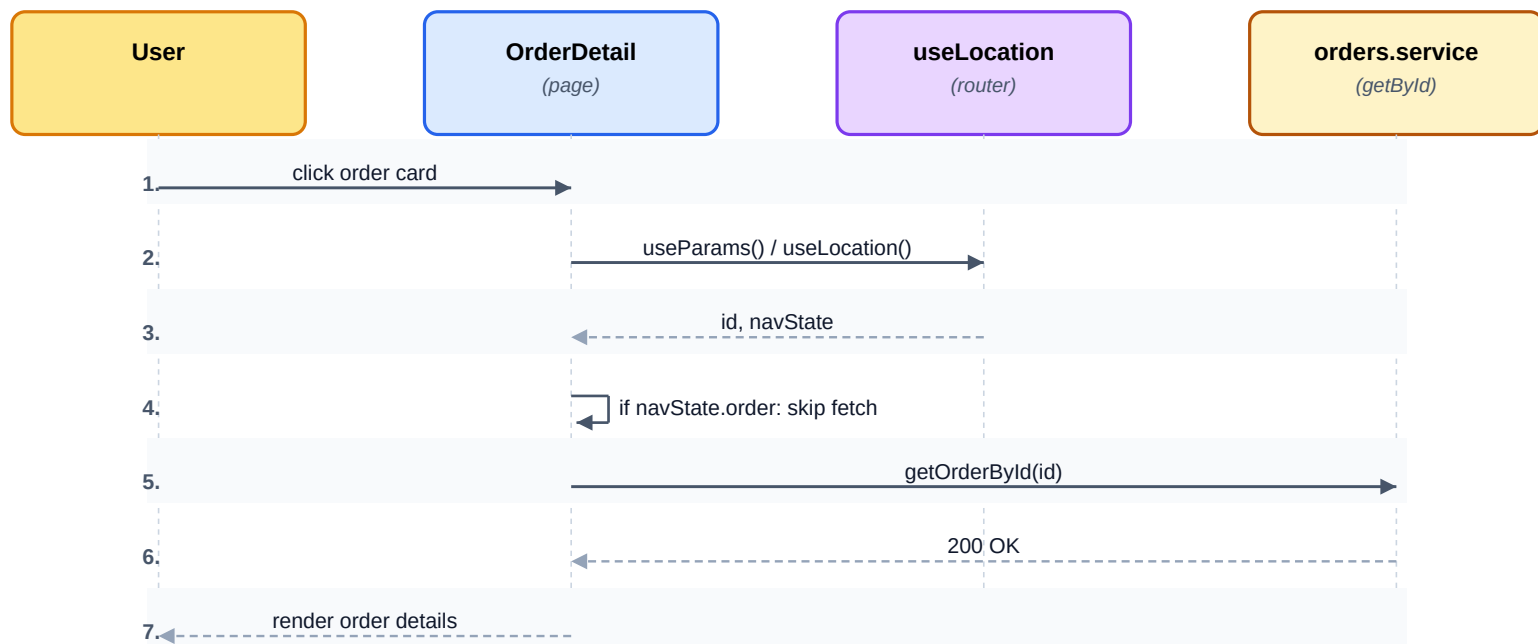


Orders list — useState + useEffect pattern. Each order card is a child component.

Data flow - order detail



Code flow - order detail (sequence view)



Order detail - reuses the order object from router state when possible; otherwise hits OrdersController.

Code

src/features/orders/hooks/useOrderHistory.js

```
import { useEffect, useState } from 'react'
import { getOrders } from '../../services/orders.service'

export function useOrderHistory() {
  const [orders, setOrders] = useState([])
  const [isLoading, setIsLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    let cancelled = false
    getOrders()
      .then((data) => { if (!cancelled) setOrders(data) })
      .catch((err) => { if (!cancelled) setError(err) })
      .finally(() => { if (!cancelled) setIsLoading(false) })
    return () => { cancelled = true }
  }, [])

  return { orders, isLoading, error }
}
```

What this code does — *useOrderHistory* — *textbook fetch with cleanup*

- Three pieces of state: orders, isLoading, error.
- Empty deps array [] means 'run once on mount'.
- cancelled flag prevents setState on an unmounted component if the user navigates away mid-fetch.
- .then().catch().finally() keeps the loading flag honest in every outcome.

Code — orders service + OrderDetail page

src/features/orders/services/orders.service.js

```
import api from '../../../api/axios.instance'

export async function getOrders() {
  const response = await api.get('/orders')
  return response.data.orders
}

export async function getOrderById(id) {
  const response = await api.get(`/orders/${encodeURIComponent(id)}`)
  return response.data.order
}

export async function placeOrder(orderData) {
  const response = await api.post('/orders', orderData)
  return response.data
}
```

src/features/orders/pages/OrderDetail.jsx (excerpt)

```
import { useParams, useLocation } from 'react-router-dom'

export default function OrderDetail() {
  const { id } = useParams()
  const { state: navState } = useLocation()
  const [order, setOrder] = useState(navState?.order ?? null)
  const [isLoading, setIsLoading] = useState(!navState?.order)

  useEffect(() => {
    if (navState?.order) return
    let cancelled = false
    getOrderById(id)
      .then((data) => { if (!cancelled) { setOrder(data); setIsLoading(false) } })
      .catch(() => { if (!cancelled) setIsLoading(false) })
    return () => { cancelled = true }
  }, [id, navState?.order])
  // ... rendering omitted
}
```

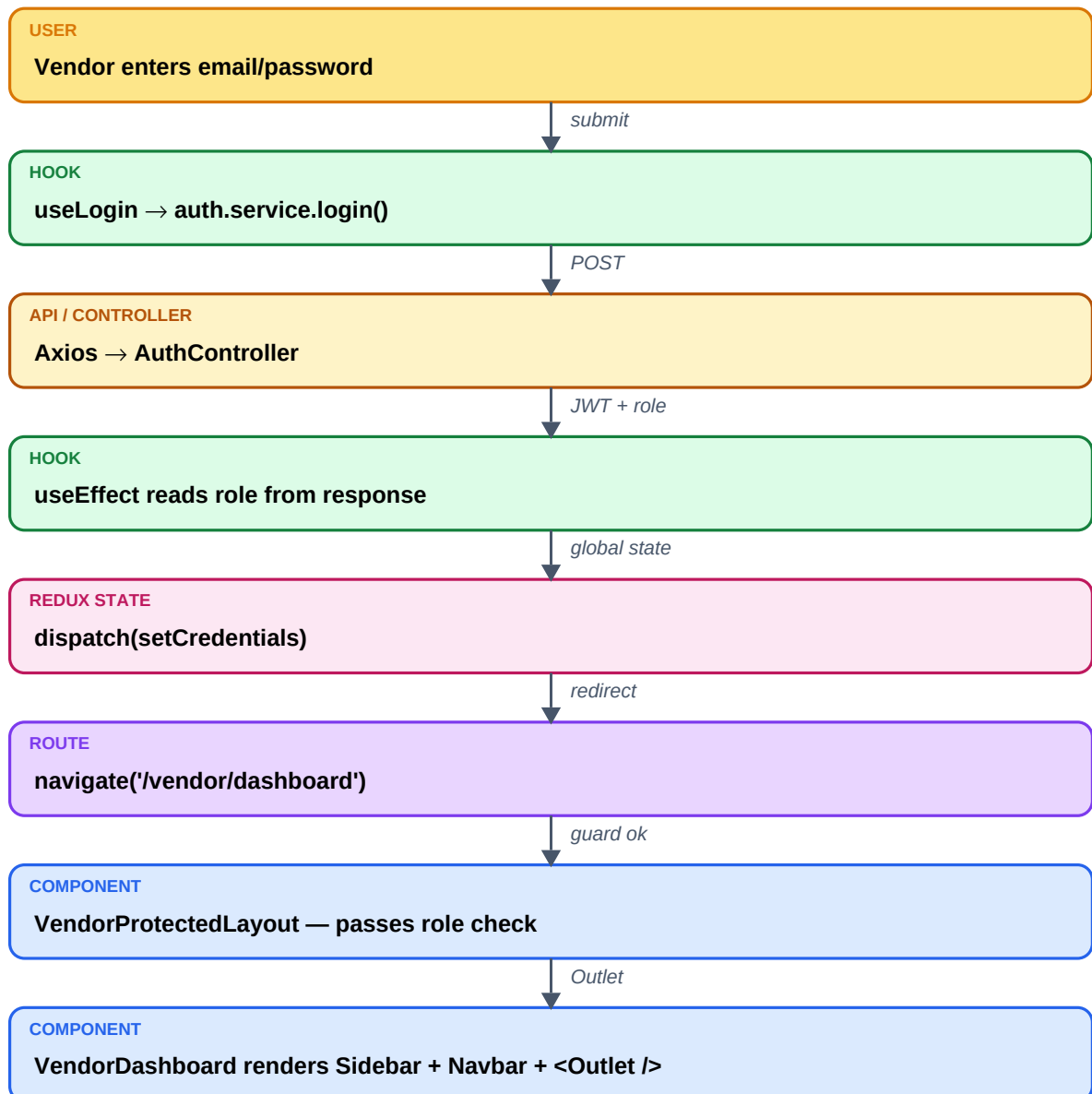
What this code does — Service + OrderDetail — reading the URL and the router state

- **useParams()** reads the `:id` segment from `/orders/:id`.
- **useLocation().state** reads anything passed via `navigate(path, { state: ... })`. After `place-order`, the order object is sitting there — so we initialize `order` with it and skip the API call.
- When the user reaches the page directly, `navState?.order` is undefined and the effect calls `OrdersController` via `getOrderById`.
- `encodeURIComponent(id)` protects against weird characters in the URL.
- The service unwraps the envelope so callers get a plain array / object.

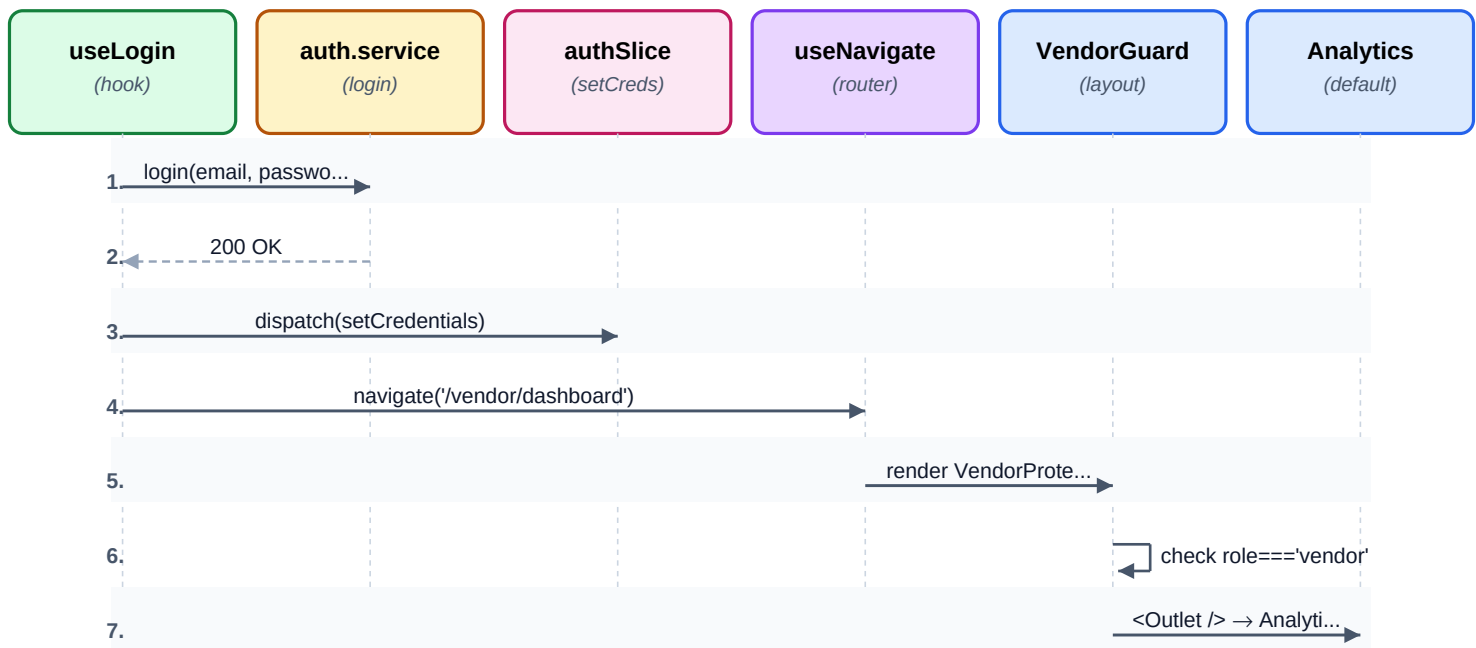
8. Vendor Flow — Step 1: Vendor Login & Role Guard

Same login form. The role on `user.role` determines the redirect. `VendorProtectedLayout` guards every vendor URL.

Data flow



Code flow — sequence view



Vendor login — same hook, but role decides redirect; the guard double-checks before letting Analytics render.

Code — role-aware redirect + guard

src/features/auth/hooks/useLogin.js (excerpt)

```
const isVendor = user.role?.toLowerCase() === ROLES.VENDOR
navigate(isVendor ? VENDOR_ROUTES.DASHBOARD : ROUTES.DASHBOARD)
```

src/router/index.jsx (vendor section)

```
function VendorProtectedLayout() {
  const isAuthenticated = useSelector(selectIsAuthenticated)
  const user = useSelector(selectCurrentUser)
  if (!isAuthenticated) return <Navigate to="/login" replace />
  if (user?.role?.toLowerCase() !== ROLES.VENDOR)
    return <Navigate to="/dashboard" replace />
  return <Outlet />
}

{ path: '/vendor', element: <VendorProtectedLayout />,
  children: [
    { index: true, element: <Navigate to="/vendor/dashboard" replace /> },
    { element: <Page component={VendorDashboard} />, children: [
      { path: 'dashboard', element: <Page component={Analytics} /> },
      { path: 'inventory', element: <Page component={InventoryManagement} /> },
      { path: 'orders', element: <Page component={OrderManagement} /> },
    ]},
  ]},
]}
```

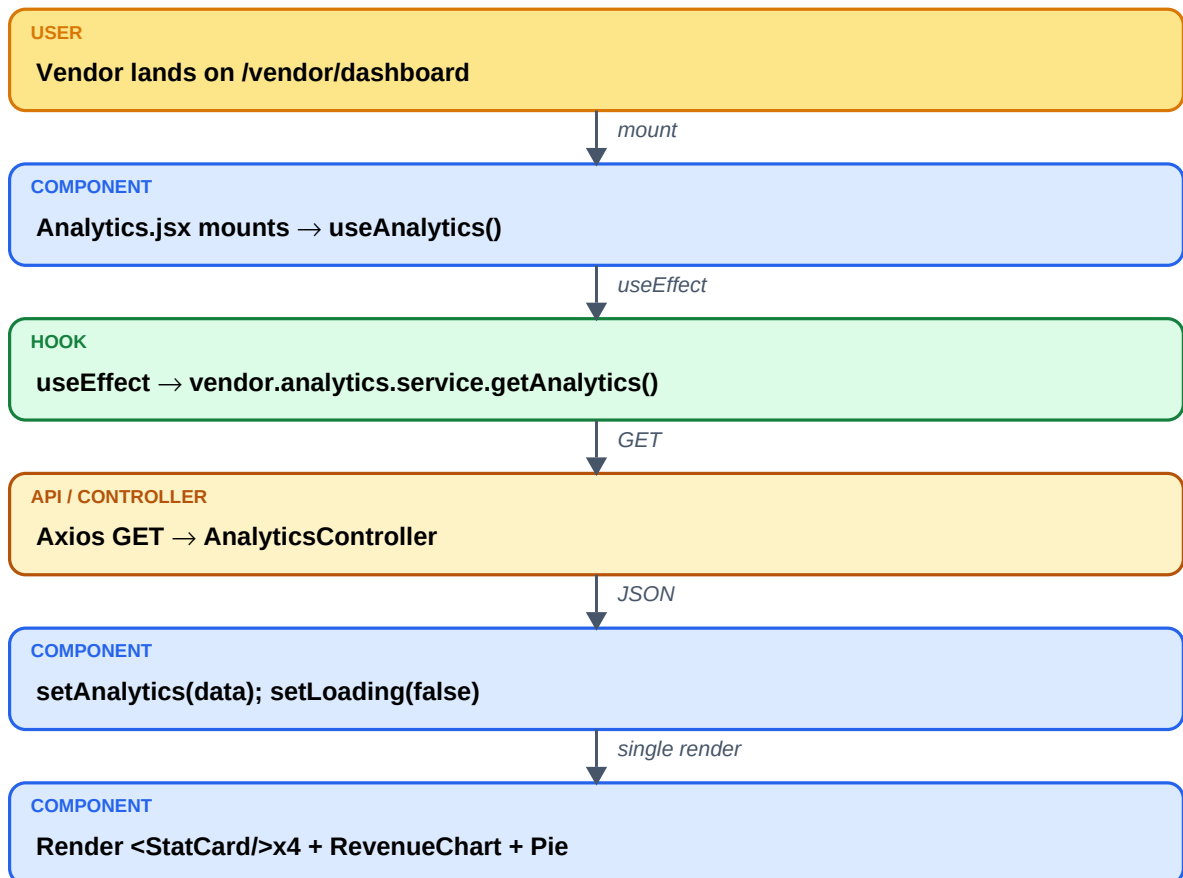
What this code does — *Layered guards + nested layouts*

- **VendorProtectedLayout** renders nothing of its own — only a redirect or an `<Outlet />`.
- Two checks: not authenticated → /login; wrong role → /dashboard. Both use `replace` so Back does not reach the protected page.
- **Pathless layout route** means `VendorDashboard` provides the sidebar+navbar layout, and its `<Outlet />` hosts the active child page.

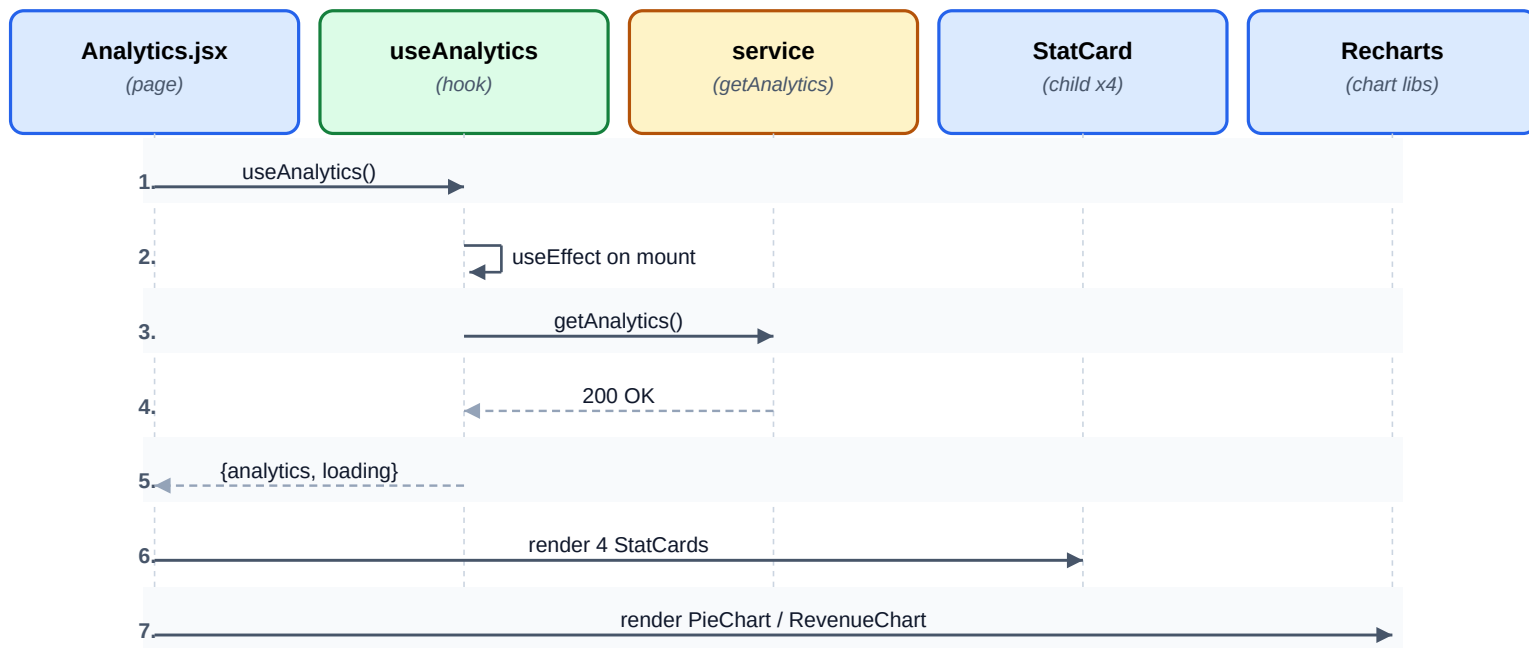
9. Vendor Flow — Step 2: Vendor Dashboard (Analytics)

Read-only screen. `useAnalytics` fetches once on mount; the page renders four stat cards and three Recharts visualisations.

Data flow



Code flow — sequence view



Analytics — single fetch, then render four cards and three charts as JSX components.

Code

src/features/vendor/hooks/useAnalytics.js

```
import { useState, useEffect } from 'react'
import { getAnalytics } from '../services/vendor.analytics.service'

export function useAnalytics() {
  const [analytics, setAnalytics] = useState(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    getAnalytics()
      .then(setAnalytics)
      .catch(() => setError('Failed to load analytics'))
      .finally(() => setLoading(false))
  }, [])

  return { analytics, loading, error }
}
```

What this code does — *useAnalytics* — *minimal fetch hook*

- Same recipe as `useOrderHistory` but slimmer because no cancel-flag is necessary here.
- `.then(setAnalytics)` = shorthand for `.then((d) => setAnalytics(d))`.
- Empty deps `[]` means 'run once on mount'. The dashboard does not refresh on its own.

Code — Analytics page (declarative charts)

src/features/vendor/pages/Analytics.jsx (excerpt)

```
import { PieChart, Pie, Cell, ResponsiveContainer } from 'recharts'

export default function Analytics() {
  const { analytics, loading, error } = useAnalytics()
  if (loading) return <Spinner.Page />
  if (error) return <p>{error}</p>

  const { totalRevenue, totalOrders, pendingOrders,
    lowStockCount, orderStatusDist } = analytics

  return (
    <div>
      <div className={styles.statsGrid}>
        <StatCard label="Total Revenue" value={`$$${totalRevenue}`} />
        <StatCard label="Total Orders" value={totalOrders} />
        <StatCard label="Pending Orders" value={pendingOrders} />
        <StatCard label="Low Stock Items" value={lowStockCount} />
      </div>

      <ResponsiveContainer width="100%" height={260}>
        <PieChart>
          <Pie data={orderStatusDist} dataKey="value" nameKey="name"
            innerRadius={60} outerRadius={100}>
            {orderStatusDist.map((entry) =>
              <Cell key={entry.name} fill={entry.color} />)}
          </Pie>
        </PieChart>
      </ResponsiveContainer>
    </div>
  )
}
```

What this code does — *Analytics — three render branches + declarative charts*

- Three early returns: **loading spinner**, **error message**, **real content**. Keeps the happy-path JSX small.
- **Object destructuring** pulls multiple fields from the API response in one line.
- **Charts are React components**. We pass `orderStatusDist` as the data prop and the chart redraws automatically.
- **`orderStatusDist.map(...)`** generates a `<Cell>` per slice — each gets a unique key (required by React for list children).
- **`ResponsiveContainer`** measures its parent and tells the chart how big to draw.

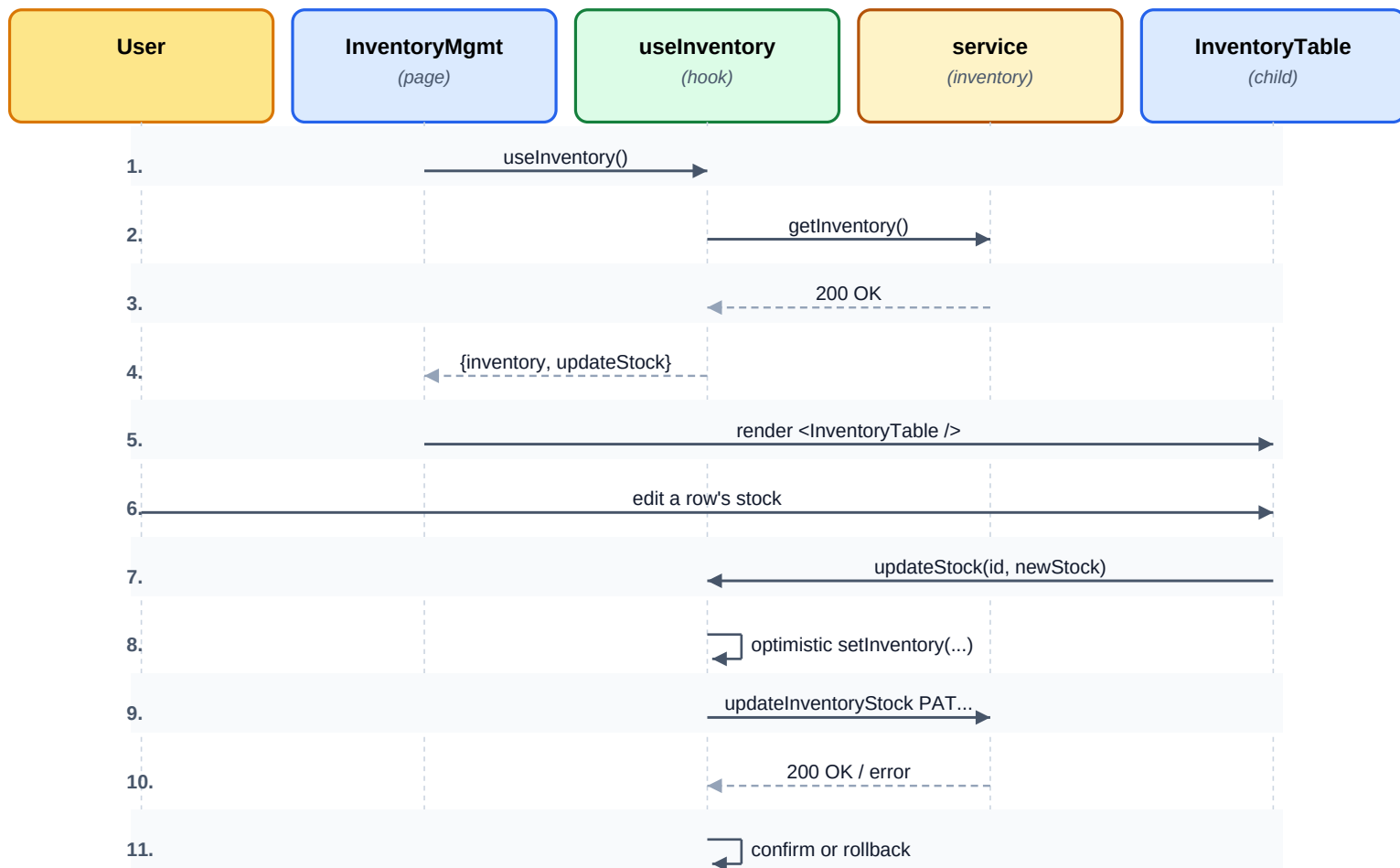
10. Vendor Flow — Step 3: Inventory Control

Inline-edit stock with **optimistic UI**: the value updates locally first, then PATCHes InventoryController; rolls back on failure.

Data flow



Code flow — sequence view



Inventory — fetch on mount, optimistic local update, PATCH, confirm-or-revert.

Code

src/features/vendor/hooks/useInventory.js

```
export function useInventory() {
  const [inventory, setInventory] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    getInventory()
      .then(setInventory)
      .catch(() => setError('Failed to load inventory'))
      .finally(() => setLoading(false))
  }, [])

  const handleUpdateStock = useCallback(async (id, newStock) => {
    const prev = inventory.find((i) => i.id === id)

    // 1. Optimistic — derive status client-side
    const status = newStock === 0 ? 'Out of Stock'
      : newStock < 5 ? 'Low Stock' : 'In Stock'
    setInventory((list) =>
      list.map((i) => i.id === id ? { ...i, stock: newStock, status } : i))

    // 2. PATCH; rollback on failure
    try {
      const updated = await updateInventoryStock(id, newStock)
      setInventory((list) =>
        list.map((i) => i.id === id ? updated : i))
    } catch (err) {
      setInventory((list) =>
        list.map((i) => i.id === id ? prev : i))
      throw err
    }
  }, [inventory])

  const lowStockCount = inventory.filter((i) => i.stock < 5).length
  return { inventory, loading, error, lowStockCount, updateStock: handleUpdateStock }
}
```

What this code does — *useInventory* — *optimistic UI + useCallback*

- **Optimistic pattern:** snapshot prev, set local state immediately, call API, replace with server copy on success or restore prev on failure.
- **setInventory((list) => ...)** uses the previous-state form to avoid stale closures.
- **useCallback** stabilizes handleUpdateStock so passing it down does not retrigger child renders.
- **{ ...i, stock: newStock }** spreads the row and overrides only one field — immutability is required for React/Redux change-detection.

Code — InventoryManagement page (filter + delegation)

src/features/vendor/pages/InventoryManagement.jsx

```
import { useState } from 'react'
import { useInventory } from '../../hooks/useInventory'
import { InventoryTable } from '../../components/InventoryTable'
import { Spinner } from '../../../shared/components/Spinner'

export default function InventoryManagement() {
  const { inventory, loading, error, lowStockCount, updateStock } = useInventory()
  const [showLowOnly, setShowLowOnly] = useState(false)

  if (loading) return <Spinner.Page />
  if (error) return <p>{error}</p>

  const displayed = showLowOnly
    ? inventory.filter((i) => i.stock < 5)
    : inventory

  return (
    <div className={styles.page}>
      <InventoryTable
        inventory={displayed}
        onUpdateStock={updateStock}
        showLowOnly={showLowOnly}
        onToggleLowOnly={() => setShowLowOnly((v) => !v)}
        lowStockCount={lowStockCount}
      />
    </div>
  )
}
```

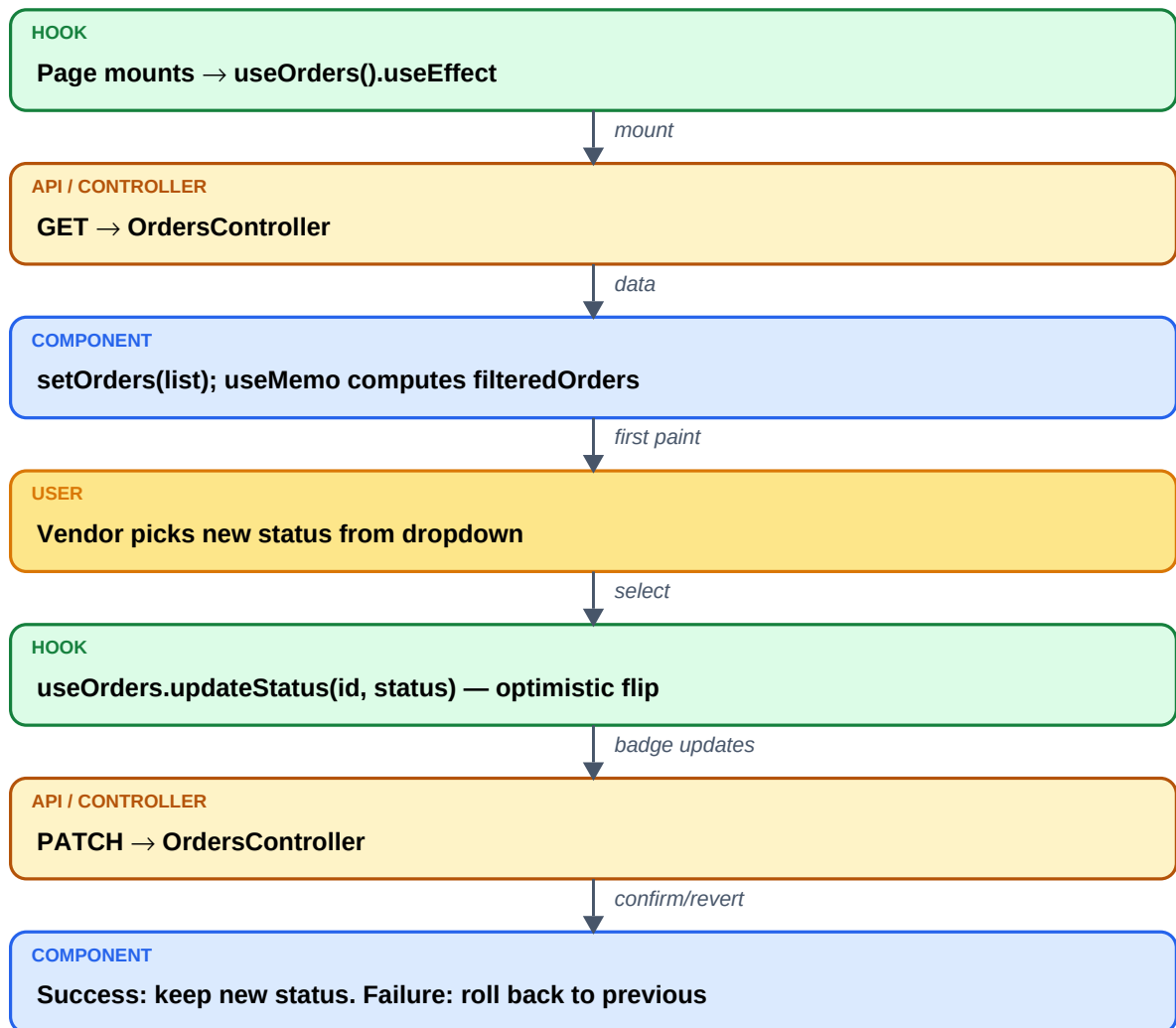
What this code does — *InventoryManagement* — page = thin shell

- Three lines pull data from the hook, one line owns the local 'show low only' toggle.
- The page does NOT know how to render the table — it delegates to `<InventoryTable>` and passes everything as props.
- **`setShowLowOnly((v) => !v)`** is the functional toggle: take the previous value and flip it.
- Local UI state (`showLowOnly`) lives in the page; data state (`inventory`) lives in the hook; both feed the table component.

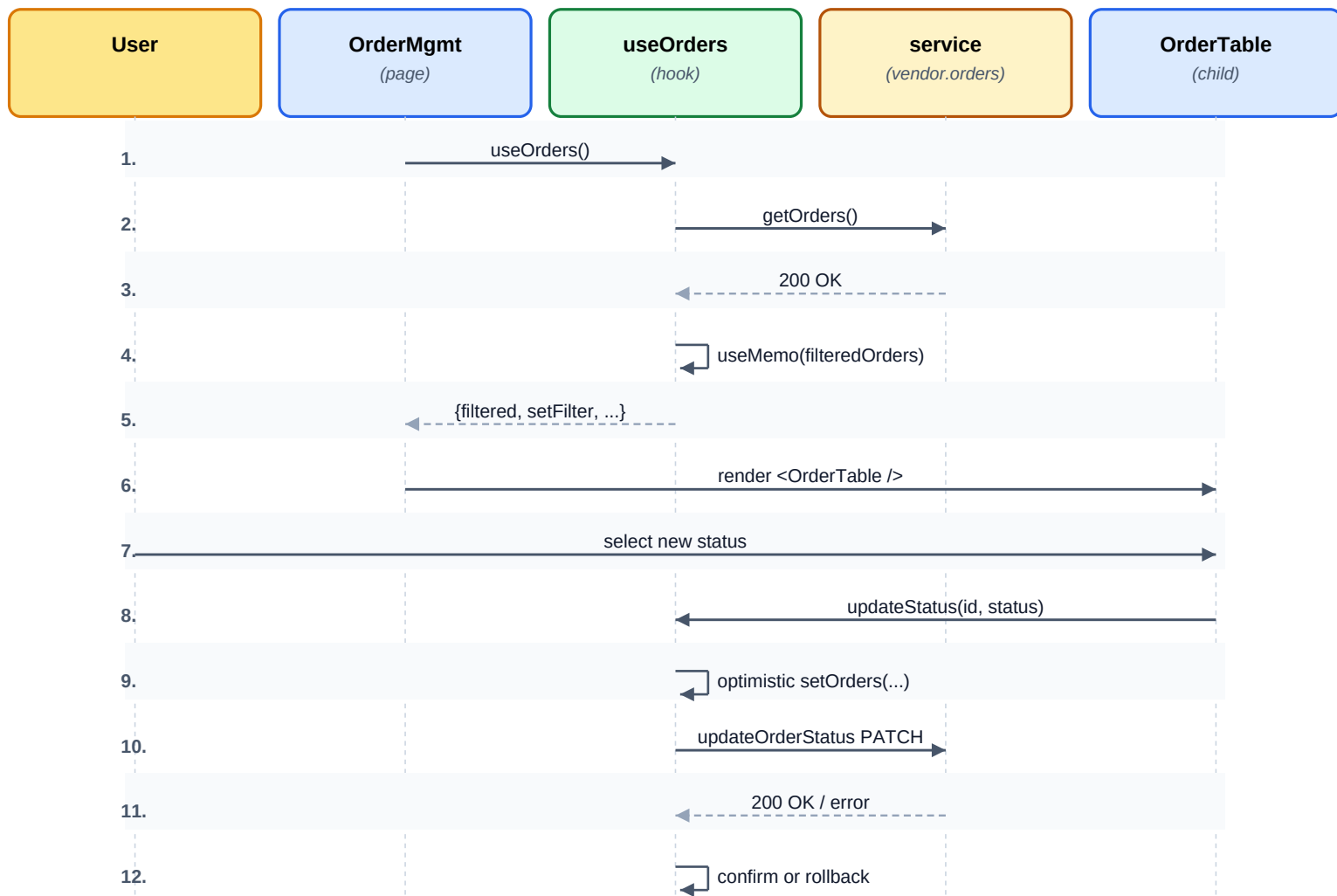
11. Vendor Flow — Step 4: Order Management

Tabs filter by status. Selecting a new status PATCHes OrdersController with optimistic UI. Same hook recipe as Inventory.

Data flow



Code flow — sequence view



Order Management — same recipe as Inventory, with one extra useMemo step for the filter.

Code

src/features/vendor/hooks/useOrders.js

```
import { useState, useEffect, useCallback, useMemo } from 'react'
import { getOrders, updateOrderStatus } from '../services/vendor.orders.service'

export const ALL_FILTER = 'All'

export function useOrders() {
  const [orders, setOrders] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [activeFilter, setFilter] = useState(ALL_FILTER)

  useEffect(() => {
    getOrders()
      .then(setOrders)
      .catch(() => setError('Failed to load orders'))
      .finally(() => setLoading(false))
  }, [])

  const filteredOrders = useMemo(
    () => activeFilter === ALL_FILTER
      ? orders
      : orders.filter((o) => o.status === activeFilter),
    [orders, activeFilter],
  )

  const handleUpdateStatus = useCallback(async (id, status) => {
    const prev = orders.find((o) => o.id === id)
    setOrders((list) => list.map((o) => o.id === id ? { ...o, status } : o))
    try {
      await updateOrderStatus(id, status)
    } catch (err) {
      setOrders((list) => list.map((o) => o.id === id ? prev : o))
      throw err
    }
  }, [orders])

  return { orders, filteredOrders, loading, error,
    activeFilter, setFilter, updateStatus: handleUpdateStatus }
}
```

What this code does — *useOrders* — *useMemo* + same optimistic recipe as *Inventory*

- **useMemo([orders, activeFilter])** caches the filtered array so children that compare props do not needlessly re-render.
- Optimistic update: snapshot, local change, PATCH, rollback on error — identical pattern to *Inventory*.
- Three layers — service / hook / page — keep concerns clean: URLs, state, JSX.

Code — OrderManagement page + service

src/features/vendor/pages/OrderManagement.jsx

```
import { useOrders } from '../hooks/useOrders'
import { OrderTable } from '../components/OrderTable'
import { Spinner } from '../../../shared/components/Spinner'

export default function OrderManagement() {
  const { filteredOrders, loading, error,
    activeFilter, setFilter, updateStatus } = useOrders()

  if (loading) return <Spinner.Page />
  if (error) return <p>{error}</p>

  return (
    <div className={styles.page}>
      <h1>Orders</h1>
      <OrderTable
        orders={filteredOrders}
        activeFilter={activeFilter}
        onFilterChange={setFilter}
        onUpdateStatus={updateStatus}
      />
    </div>
  )
}
```

src/features/vendor/services/vendor.orders.service.js

```
import api from '../../../api/axios.instance'

export const ORDER_STATUSES = ['Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled']

export async function getOrders() {
  const response = await api.get('/vendor/orders')
  return response.data
}

export async function updateOrderStatus(id, status) {
  const response = await api.patch(`/vendor/orders/${id}/status`, { status })
  return response.data
}
```

What this code does — OrderManagement + service — three layers, one job each

- **Service** knows the URL and shape (/vendor/orders, /vendor/orders/{id}/status).
- **Hook** (useOrders) owns the state, filtering, and optimistic updates.
- **Page** is a thin shell — imports the hook + the table and wires them together.
- **Component** (OrderTable) is purely presentational. It does not know whether the data came from Redux, a hook, or a JSON file.

12. Cheat-Sheet — Every Concept in One Line

Component	A function returning JSX. Smallest building block.
Props	Inputs from parent. Read-only inside the child.
State (useState)	Per-component memory; setting it triggers a re-render.
Effect (useEffect)	Runs AFTER paint — fetching, navigation, logging.
Custom Hook	Reusable function starting with 'use' that bundles hooks together.
useMemo	Caches a computed value until its dependencies change.
useCallback	Caches a function reference so children skip re-renders.
Context	Share a value with descendants without prop-drilling.
Redux Slice	Global app state — auth and cart in this project.
useSelector	Read a slice value and subscribe to changes.
useDispatch	Send an action to mutate a slice.
Reducer	(state, action) => newState. Pure function.
Router	Maps URL → component (createBrowserRouter).
Outlet	Placeholder where child routes render inside a layout.
Navigate	Imperative redirect element.
useParams	Reads dynamic URL segments like :id.
useLocation	Reads URL state passed via navigate().
useNavigate	Hook returning a function to change the URL.
lazy + Suspense	Code-splits a page; shows fallback while loading.
ErrorBoundary	Class component catching render errors.
useActionState	React 19 form hook: [state, formAction, isPending].
React Hook Form	Library to manage form fields and validation cleanly.
Axios + Interceptors	Single instance with auto JWT + error normalization.
Optimistic UI	Update local state first, roll back on API failure.
Sequence Diagram	Lanes per file + horizontal arrows showing function-call order.

Data-Flow Diagram

Vertical chain showing who triggers what across roles (user/comp/hook/state/api/route).

Suggested study path. Pick one chapter per day. For each chapter, read the **data-flow diagram** first to get the high-level idea, then study the **code-flow sequence diagram** while opening the named files in VS Code. Within a week you will be able to follow any feature in this codebase by jumping between files — no IDE search required.